SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	VIVETHA B	**Reg. No.:**	192424170
Section / Batch:	B.TECH AI&DS	**Date:**	

Code of Conduct

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:VIVETHA B

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

A language processing system can represent sentence structure using either **Context-Free Grammar (CFG) trees** or **Dependency Parsing**. Both approaches are useful, but they represent relationships in different ways.

Context-Free Grammar (CFG) Trees

CFG represents a sentence using **hierarchical phrase structures**.

For example:

"The student reads a book."

A CFG can represent it as:

S → NP VP

NP → Det N

VP → V NP

This shows that:

Sentence → Noun Phrase + Verb Phrase

Noun Phrase → The + student

Verb Phrase → reads + Noun Phrase

Noun Phrase → a + book

The main strength of CFG is that it clearly represents **constituents or phrases** and shows how words combine to form larger grammatical units.

CFG is useful for:

- Grammar checking
- Phrase structure analysis
- Parse tree generation
- Syntactic analysis
- Understanding sentence hierarchy

However, CFG trees can become complicated for long sentences because they contain many intermediate phrase nodes.

Dependency Parsing

Dependency parsing focuses directly on the **relationships between individual words**.

For:

"The student reads a book."

the dependency relationships can be represented as:

reads → student

reads → book

student → The

book → a

Here, **reads** is the main word or root.

The parser directly identifies that:

student → subject of reads

book → object of reads

This makes dependency parsing particularly useful for understanding **who performed an action and what the action affected**.

Dependency parsing is useful for:

- Information extraction
- Semantic analysis
- Question answering
- Relation extraction
- Machine translation
- Chatbots
- Understanding long and complex sentences

Main Difference

CFG primarily asks:

"How are words grouped into phrases?"

Dependency parsing primarily asks:

"How are words grammatically related to each other?"

For example, CFG emphasizes:

S → NP + VP

while dependency parsing emphasizes:

reads → student

reads → book

Which Is More Effective for Capturing Word Relationships?

Dependency parsing is generally more effective for directly capturing relationships between words.

For example, in:

"The doctor prescribed medicine to the patient."

dependency parsing can directly identify:

doctor → subject/agent

prescribed → main verb

medicine → object

patient → recipient

This is useful when an NLP system needs to extract relationships between entities and actions.

However, **CFG is better for representing hierarchical phrase structure** and checking whether a sentence follows grammatical rules.

Conclusion

CFG trees are better for representing **hierarchical syntactic structure and phrase organization**, while **dependency parsing** is better for representing **direct grammatical relationships between words**.

Therefore, for a system whose main goal is to **capture relationships between words**, **dependency parsing is the more effective approach**. For complete grammar checking and phrase-level syntactic analysis, CFG can still be valuable.

Best practical solution: Use **CFG and dependency parsing together**—CFG for phrase structure and dependency parsing for direct word-to-word relationships.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.
Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

A real-time application must process user input quickly, even when the input is **incomplete, ambiguous, or complex**. Top-down parsing and Earley parsing approach this problem differently.

Top-Down Parsing

Top-down parsing starts from the **start symbol of the grammar** and tries to generate the input sentence by applying grammar rules.

For example, for:

"Book a flight to Delhi..."

the parser starts with:

S → VP

and then expands the grammar rules until it tries to match the input.

Its advantages are:

- Simple to understand and implement
- Works well with small and simple grammars
- Efficient when the sentence structure is predictable
- Suitable for simple applications

However, it has important limitations for dynamic input.

Backtracking: If the parser selects an incorrect rule, it may need to go backward and try another rule.

Ambiguity: Multiple possible interpretations can cause the parser to explore several alternatives.

Incomplete input: If the user stops at:

"Book a flight to..."

the parser may not have enough information to complete the expected structure.

Real-time performance: Repeated backtracking can increase response time.

Earley Parsing

Earley parsing is a **dynamic-programming parsing technique** that can handle general Context-Free Grammars.

Instead of repeatedly starting over when different possibilities occur, it stores intermediate parsing states and reuses them.

For example, with:

"Book a flight to Delhi with..."

Earley parsing can maintain partial parsing information while waiting for the remaining input.

Its advantages include:

- Handles ambiguous sentences
- Handles incomplete or partial input
- Reduces repeated parsing work through dynamic programming
- Works with general CFGs
- Can handle left-recursive grammar rules
- Maintains multiple possible interpretations

This makes it particularly useful for applications such as **voice assistants, chatbots, search systems, and interactive NLP systems**.

Difference in Working

Top-Down Parsing:

Start Symbol → Grammar Expansion → Match Input → Backtrack if Necessary

Earley Parsing:

Input → Predict → Scan → Complete → Store Partial States → Continue Parsing

The major difference is that **top-down parsing may repeatedly explore alternatives**, whereas **Earley parsing stores and reuses partial results**.

Which Is More Suitable?

For simple and predictable sentences, **top-down parsing can be sufficient**.

For dynamic applications where users may provide:

Incomplete input

Ambiguous sentences

Long sentences

Conversational expressions

Real-time commands

Earley parsing is more suitable.

For example:

"Transfer money to..."

The input is incomplete. Earley parsing can maintain the partial parse while additional words arrive, whereas a traditional top-down parser may require more backtracking or may fail until the input is complete.

Conclusion

Top-down parsing is simple and effective for small, predictable grammars, but it can suffer from **backtracking and difficulty with ambiguity and incomplete input**.

Earley parsing is more flexible because it uses **dynamic programming, supports ambiguity, and can maintain partial parsing states**.

Therefore, for a **real-time application with dynamic, incomplete, and ambiguous user input, Earley parsing is the better choice** because it provides greater robustness and flexibility while avoiding unnecessary repeated parsing.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.
Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

The sentence is:

“She saw the man with a telescope.”

This sentence is ambiguous because the phrase **“with a telescope”** can have two different meanings.

Interpretation 1: She used a telescope to see the man.

Meaning:
She saw [the man] [with a telescope].

Here, **"with a telescope"** describes the instrument used by **She**.

Interpretation 2: The man had a telescope.

Meaning:
She saw [the man [with a telescope]].

Here, **"with a telescope"** describes the **man**.

Therefore, the system needs to determine which interpretation is most appropriate based on context.

1. Context-Free Grammar (CFG)

CFG represents sentences using grammar rules.

For example:

S → NP VP

VP → V NP

NP → Det N

PP → P NP

CFG can generate both possible parse structures for the sentence.

However, a basic CFG does not assign probabilities to the different interpretations.

Therefore, it may produce:

Parse 1 → She used the telescope

and:

Parse 2 → The man had the telescope

without knowing which one is more likely.

Advantages:

- Simple and interpretable
- Good for representing grammatical structure
- Useful for grammar checking
- Easy to construct for limited domains

Limitation:

CFG cannot naturally determine the most likely interpretation when multiple valid parse trees exist.

2. Probabilistic Context-Free Grammar (PCFG)

PCFG extends CFG by assigning probabilities to grammar rules.

For example:

VP → V NP PP [0.6]

NP → NP PP [0.4]

The system calculates the probability of different parse trees and selects the more probable interpretation.

Therefore, PCFG can resolve some ambiguity better than a basic CFG.

For example, if training data shows that **"saw the man with a telescope"** more frequently means that the person used a telescope, the corresponding parse can receive a higher probability.

Advantages:

- Handles ambiguity better than CFG
- Uses statistical information
- Selects the most probable parse
- Relatively interpretable

Limitations:

- Depends on the quality of training data
- Basic PCFG mainly uses grammatical probabilities
- May not understand deeper semantic or contextual information

3. Neural Parsing

Neural parsers use machine-learning models, particularly modern contextual language models, to understand sentence structure.

They can use information from the **entire sentence and surrounding context**.

For example:

"She saw the man with a telescope because she was observing the building."

The surrounding context strongly suggests that **she used the telescope**.

But:

"She saw the man with a telescope. He was carrying it."

suggests that **the man had the telescope**.

Neural parsing can use these contextual clues to select the appropriate interpretation.

Advantages:

- Handles complex ambiguity
- Uses wider contextual information

- Can learn linguistic patterns automatically
- Works well with large-scale real-world language
- Can integrate syntactic and semantic information

Limitations:

- Requires large amounts of training data
- Computationally expensive
- Less interpretable than CFG
- Can sometimes produce incorrect interpretations

Comparison

CFG:

Focuses mainly on **grammatical structure**.

Ambiguity handling: Low

Interpretability: Very high

Data requirement: Low

Real-world flexibility: Limited

PCFG:

Focuses on **grammar + probability**.

Ambiguity handling: Better than CFG

Interpretability: High

Data requirement: Moderate

Real-world flexibility: Moderate

Neural Parsing:

Focuses on **grammar + context + learned representations**.

Ambiguity handling: High

Interpretability: Lower

Data requirement: High

Real-world flexibility: High

Which Is Most Effective?

For **real-world NLP applications**, **neural parsing is generally the most effective** of the three because real-world language contains complex ambiguity, contextual dependencies, informal expressions, and long-distance relationships.

However, this does not mean CFG and PCFG are useless.

A practical NLP system can combine them:

Neural Model → Context Understanding

PCFG → Probabilistic Syntactic Analysis

CFG → Grammar Constraints and Interpretability

This hybrid approach can provide both **accuracy and structural reliability**.

Conclusion

For the sentence **"She saw the man with a telescope,"** a **CFG** can generate multiple valid interpretations but cannot determine which is correct. A **PCFG** can rank the interpretations using probabilities and select the most likely one. **Neural parsing** can use broader contextual information to determine the intended meaning more effectively.

Therefore, **neural parsing is the most suitable for modern real-world applications**, while combining neural methods with traditional CFG/PCFG techniques can provide better **accuracy, interpretability, and robustness**.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.
Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

A grammar system must handle two important problems: **subject–verb agreement** and **verb argument structures**. Feature structures and subcategorization frames address these problems in different ways.

1. Feature Structures

Feature structures represent grammatical properties of words using features and values.

Common features include:

Number → Singular / Plural

Person → First / Second / Third

Tense → Present / Past

For example:

She

Person = Third

Number = Singular

The verb:

runs

Person = Third

Number = Singular

The features match:

She + runs → Correct

But:

She + run → Incorrect

because the subject is singular while the verb form is not compatible with the required agreement.

Feature structures are therefore very useful for enforcing **grammatical agreement**.

They can also represent other properties such as:

Gender

Case

Tense

Person

Number

2. Subcategorization Frames

Subcategorization frames describe the **types and number of arguments or complements that a verb requires**.

For example:

eat → Subject + Verb + Object

Sentence:

The patient eats medicine.

Here, **eat** expects an object.

Another example:

give → Subject + Verb + Object + Recipient

Sentence:

The doctor gave the patient medicine.

The verb **give** requires:

Agent → Doctor

Recipient → Patient

Theme → Medicine

Similarly:

sleep → Subject + Verb

Sentence:

The patient sleeps.

The verb does not normally require a direct object.

Subcategorization frames are therefore mainly concerned with **verb argument structure**, rather than subject–verb agreement.

3. Main Difference

Feature Structures answer:

"Do the grammatical features of the words agree?"

For example:

He runs → Correct

They runs → Incorrect

Subcategorization Frames answer:

"What arguments or complements does this verb require?"

For example:

The doctor prescribed medicine.

The verb **prescribe** expects a person/agent and something being prescribed.

4. Which Provides Better Support for Grammatical Correctness?

If the main requirement is **subject–verb agreement**, **Feature Structures provide better support**.

For example:

She writes → Correct

They write → Correct

She write → Incorrect

Feature structures can explicitly compare the **Person** and **Number** features of the subject and verb.

However, if the system needs to verify whether a verb has the correct **arguments and complements**, subcategorization frames are more suitable.

For example:

The doctor prescribed medicine to the patient.

The system can verify that the verb **prescribed** has the expected arguments.

5. Best Practical Solution

For a complete grammar system, the best approach is to **combine both methods**.

Feature Structures → Agreement Checking

Subcategorization Frames → Verb Argument Checking

For example:

"The doctors prescribes medicine."

Feature structures detect:

doctors → Plural

prescribes → Singular

Therefore:

Agreement Error

For:

"The doctor prescribed."

the subcategorization system can determine whether **prescribed** requires an object in the particular usage and identify a potentially incomplete structure.

Conclusion

Feature structures are better for enforcing **subject–verb agreement and grammatical feature compatibility**, while **subcategorization frames** are better for handling **verb argument structures and required complements**.

Therefore, neither approach completely replaces the other. For a robust grammar system, the best solution is:

Feature Structures + Subcategorization Frames

This combination provides better **grammatical correctness, syntactic analysis, and accurate understanding of verb–argument relationships**.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.
Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

A large-scale NLP system needs to process a huge number of sentences quickly while maintaining good parsing accuracy. **Transition-based parsing** and **graph-based parsing** use different approaches to construct dependency structures.

1. Transition-Based Parsing

Transition-based parsing builds the dependency tree **step by step** using a sequence of parsing actions.

Common actions include:

SHIFT → Move a word into the parsing structure.

REDUCE → Remove a completed element.

LEFT-ARC → Create a dependency from one word to another.

RIGHT-ARC → Create a dependency in the opposite direction.

For example, in:

"The student reads a book."

the parser gradually processes the words and creates relationships such as:

reads → student

reads → book

The next action is usually selected using a **classifier or neural model**.

Advantages of Transition-Based Parsing

- Very fast parsing
- Usually requires approximately linear time
- Suitable for real-time applications
- Low memory requirements
- Efficient for very large datasets
- Can process sentences incrementally

Limitations

The main limitation is **local decision-making**.

An incorrect decision made early in the parsing process can affect later decisions. This is sometimes called **error propagation**.

For example, if the parser incorrectly identifies the dependency of a word early in the sentence, subsequent transitions may build on that incorrect structure.

2. Graph-Based Parsing

Graph-based parsing treats dependency parsing as a **global optimization problem**.

The sentence is represented as a graph in which words are nodes and possible dependency relationships are edges.

The parser assigns scores to possible dependency edges and searches for the dependency tree with the **highest overall score**.

For example:

reads → student

reads → book

can be scored as possible dependency relationships.

The parser then selects the combination of edges that produces the best complete dependency tree.

Advantages of Graph-Based Parsing

- Makes globally informed decisions
- Considers relationships across the entire sentence
- Less affected by early local errors
- Often provides strong parsing accuracy
- Useful for complex syntactic structures

Limitations

- Computationally more expensive
- Requires more memory
- Global optimization can be slower
- Less suitable for extremely high-throughput or real-time processing

3. Difference in Decision-Making

The main difference is:

Transition-Based Parsing → Local, sequential decisions

The parser decides:

"What action should I take next?"

Graph-Based Parsing → Global optimization

The parser decides:

"Which complete dependency tree has the highest score?"

Therefore, transition-based parsing focuses on **speed and sequential decisions**, while graph-based parsing focuses on **global consistency and accuracy**.

4. Performance Comparison

For **speed**, transition-based parsing generally has the advantage because it can process sentences with approximately linear-time transition sequences.

For **memory usage**, transition-based parsing is generally more efficient.

For **global accuracy**, graph-based parsing can have an advantage because it evaluates the dependency structure more globally.

For **large-scale datasets**, however, processing speed and resource efficiency are extremely important.

5. Which Is More Suitable for Large-Scale Applications?

For a large-scale NLP system processing **millions of sentences**, **transition-based parsing is generally more suitable** when fast processing and scalability are the primary requirements.

For example, applications such as:

Search engines

Chatbots

Social media analysis

Real-time translation

Large document processing

may benefit from the speed and low computational cost of transition-based parsing.

However, if the application prioritizes **maximum parsing accuracy over speed**, particularly for complex sentences, graph-based parsing may be preferable.

Conclusion

Transition-Based Parsing uses sequential local decisions and is generally **faster, more memory-efficient, and easier to scale**.

Graph-Based Parsing evaluates possible dependency structures more globally and can provide **stronger global consistency**, but it usually requires greater computational resources.

Therefore, for a **large-scale NLP system where millions of sentences must be processed quickly**, **transition-based parsing is the more suitable choice** because of its **high speed, lower resource requirements, and scalability**.

A practical industry system can also use **neural transition-based parsing** to achieve a strong balance between **accuracy and processing speed**.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____